

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка

Кафедра програмування

Звіт

Про виконану роботу з курсу Програмної інженерії

Проект «PetSearchHome»

Заняття № 11

Львів – 2026

Тема: вдосконалення веб-застосунку

Завдання:

- Реалізувати по 1 юз-кейсу на учасника команди з юніт тестами.
- Додати інтеграцію із зовнішнім API на ваш вибір. Використати typed clients або generated clients для цього
- Написати власний RateLimiting action фільтр. Дозволити не більше N запитів за хвилину з певної IP. Перенаправити користувача на сторінку з помилкою, якщо кількість запитів перевищено. Застосувати фільтр як атрибут на довільних трьох запитах

Для виконання поставлених завдань роботу було розподілено між учасниками команди таким чином:

- **Лук'янчук Денис** – реалізував use-case Змінити пароль, Написав власний RateLimiting action фільтр.
- **Денисюк Денис** – реалізував use-case Очистити історію, додав інтеграцію із зовнішнім API
- **Патрило Павло** – реалізував use-case Видалити акаунт, написання звіту про виконану роботу
- **Бойко Максим** – реалізував use-case Надсилати сповіщення, написання тестів.
- **Левус Любомир** – написання тестів, реалізував use-case «Забули пароль»

1. Реалізовані use-case

1.1. Сповіщення в чатах

Реалізовано механізм сповіщень для чатів між користувачами. Коли користувач отримує нове повідомлення у чаті, система автоматично формує сповіщення. Сповіщення зберігаються в базі даних та відображаються у відповідному розділі інтерфейсу.

Логіка сповіщень реалізована на рівні сервісу, контролер обробляє запити на отримання та позначення сповіщень як прочитаних. Написано юніт-тести для перевірки коректності формування та доставки сповіщень у різних сценаріях.

1.2. Очистити історію чату

Реалізовано use-case очищення історії чату. Користувач має можливість видалити всі повідомлення у певному чаті зі свого боку. Після підтвердження дії — відповідні записи позначаються як видалені в базі даних, а інтерфейс оновлюється без перезавантаження сторінки.

Операція очищення є оборотною лише на рівні адміністратора; для звичайного користувача видалені повідомлення більше не відображаються. Сервіс та репозиторій протестовано юніт-тестами.

1.3. Забули пароль

Реалізовано стандартний flow відновлення доступу до акаунту. Користувач вводить свою електронну адресу, система перевіряє її наявність і надсилає

лист із посиланням для скидання пароля. Посилання містить унікальний токен з обмеженим терміном дії.

Після переходу за посиланням відкривається форма для введення нового пароля. Після успішного скидання токен анулюється. Реалізовано валідацію вхідних даних та логування всіх кроків процесу. Написано юніт-тести для сервісу відновлення пароля.

1.4. Видалення акаунту

Реалізовано можливість видалення власного акаунту користувачем. Перед остаточним видаленням система запитує підтвердження дії. Після підтвердження — акаунт деактивується, пов'язані оголошення приховуються з публічного каталогу, сесія завершується та виконується редирект на головну сторінку.

Видалення реалізовано через soft delete — записи позначаються як видалені, а не фізично прибираються з бази. Написано юніт-тести для відповідного сервісу.

1.5. Зміна пароля з профілю

Реалізовано use-case зміни пароля з розділу налаштувань профілю. Користувач вводить поточний пароль та новий пароль (з підтвердженням). Система перевіряє коректність поточного пароля перед збереженням нового.

Реалізовано валідацію: новий пароль повинен відповідати вимогам складності, а поля «новий пароль» і «підтвердження» мають збігатися. У разі помилки — відображаються відповідні повідомлення. Написано юніт-тести, що покривають успішні та негативні сценарії.

2. Написання власного RateLimiting action фільтра

Реалізовано власний action-фільтр RateLimitingAttribute, розташований у PetSearchHome_WEB/Filters/RateLimitingAttribute.cs. Фільтр обмежує кількість запитів до певного action з однієї IP-адреси протягом визначеного часового вікна.

Принцип роботи:

- Ліміт запитів визначається за IP-адресою клієнта у вікні тривалістю 1 хвилина.
- Лічильник запитів зберігається в пам'яті (in-memory) з прив'язкою до IP та імені action.
- При перевищенні встановленого ліміту — виконується redirect на сторінку Home/RateLimitExceeded.
- Сторінка помилки PetSearchHome_WEB/Views/Home/RateLimitExceeded.cshtml повертає HTTP-статус 429 (Too Many Requests).

Фільтр застосовано як атрибут на трьох endpoints:

- Home/Index — PetSearchHome_WEB/Controllers/HomeController.cs:46
- Home/Details —
PetSearchHome_WEB/Controllers/HomeController.cs:88
- Account/Login (POST) —
PetSearchHome_WEB/Controllers/AccountController.cs:63

Застосування фільтра на сторінці входу (Login) додатково захищає систему від атак перебору паролів (brute force). Фільтр реалізований як атрибут, що дозволяє легко підключати його до будь-якого action контролера без змін у бізнес-логіці.

3. Інтеграція із зовнішнім API

Денисюк Денис реалізував інтеграцію із зовнішнім API з використанням підходу typed client у ASP.NET Core. Typed client — це клас, що інкапсулює логіку HTTP-запитів до конкретного зовнішнього сервісу та реєструється в DI-контейнері.

Взаємодія з API реалізована через відповідний сервіс, який інжектуються в контролер. Це забезпечує розділення відповідальностей та спрощує тестування за допомогою моків.

4. Тестування

До всіх реалізованих use-case написано юніт-тести з використанням xUnit. Тести охоплюють:

- позитивні сценарії — коректне виконання основної логіки;
- негативні сценарії — обробка некоректних вхідних даних, неіснуючих сутностей, порушень бізнес-правил.

При написанні тестів дотримано принципів ізоляції (використання моків для залежностей), читабельності та незалежності тестів між собою. Бойко Максим та Левус Любомир забезпечували загальне покриття тестами функціональності, реалізованої всіма учасниками команди.

Висновок

У ході виконання роботи команда реалізувала п'ять use-case: сповіщення в чатах, очищення історії чату, відновлення пароля через email, видалення акаунту та зміну пароля з профілю. Додатково розширено функціональність адміністративного модуля модерації скарг — адміністратор тепер може переглядати оголошення безпосередньо зі сторінки скарги та переходити до нього.

Реалізовано власний RateLimiting action-фільтр, що захищає систему від надмірного навантаження та атак перебору на рівні IP-адреси. Виконано інтеграцію із зовнішнім API з використанням typed client і HttpClientFactory. До всіх реалізованих компонентів написано юніт-тести, що підтверджують коректність роботи як у штатних, так і в граничних сценаріях